企业工单与审批系统 —— 项目总结与面试复盘

个人项目 · 实习面试复盘 · 生成日期 2026-09-22

> 这份文档回答三个问题：
>
> 1. 这个项目现在够不够当一段实习简历经历？
> 2. 这个系统到底是怎么做出来的（从建单到办结的完整链路）？
> 3. 面试官会问什么、我该怎么答，哪些地方会被问穿？

一、先给结论：够不够当项目经历

够，但只够"讲得清楚"的那部分。 判断标准不是代码行数，而是面试官能不能在 15 分钟里 问出你真正做过、真正想过的痕迹。用这个标准量一下：

维度	现在的情况	面试价值
业务完整度	建单 → 提交 → 多级审批 → 驳回/撤回/作废 → 办结，闭环	高。能讲清"一张单的一生"
技术栈匹配度	Spring Boot 3 + MyBatis-Plus + MySQL + JWT + Vue 3，就是实习岗常见组合	高
有真实复杂度	并发单号、审批链解析、状态机、权限三道闸、流式导出、审计日志	**这是最值钱的部分**
工程化	234 个后端测试 + 6 个浏览器端到端脚本 + 迁移脚本	中上，超过多数学生项目
规模	后端 108 个类 / 6360 行，测试 4019 行；前端 29 个文件 / 4062 行	够用，不算大
部署	本机跑，没有 Docker、没有 CI、没有线上环境	**短板，会被问**
中间件深度	没有 Redis、没有消息队列、没有分库分表	短板，但对实习生不是硬伤

一句话定位：这是一个"业务闭环 + 有若干真实工程取舍"的单体项目， 适合应聘 Java 后端 / 全栈实习。它的竞争力不在"技术多新"，而在**每个设计都能说出 为什么不那样做**——这正是大多数培训班项目缺的东西。

要避开的风险：如果面试时只能说出"用了 Spring Boot + Vue 做了个工单系统"， 那它就退化成和所有学生项目一样；如果能把下面第三章的七个点讲透任意三个， 它就能撑起整场面试。

二、一页纸项目总结（可背）

> **企业工单与审批系统**：面向企业内部的工单流转与审批平台，支持员工提单、
> 按工单类型配置多级审批链、审批人处理待办、申请人修改重提、撤回与作废，
> 并提供站内通知、工单讨论、附件管理、操作审计、数据看板与 Excel 全量导出。
> 后端用 **Spring Boot 3 + MyBatis-Plus + MySQL 8 + JWT**，前后端分离；
> 前端用 **Vue 3 + Vite + Element Plus + Pinia**。系统共 10 张表、45 个 REST 接口，
> 后端 108 个类约 6300 行，前端约 4000 行。
> 我负责全部设计与实现。技术上重点解决了四类问题：
> 一是**审批流的可配置与正确性**（审批链按角色占位符解析，自动规避自审、同人去重、
> 主管缺失时向上追溯或兜底到管理员）；
> 二是**并发一致性**（工单单号用数据库原子自增代替 `MAX+1`，
> 20 并发实测从只有 3 个成功修到全部成功；催办做行锁 + 30 分钟限流）；
> 三是**权限与审计**（Spring Security 接口级 + 角色级 + Service 数据级三道闸；
> 审批记录与操作日志分开建模，一份给程序算状态、一份给人看历史）；
> 四是**大数据量导出**（用 POI 的 SXSSF 流式写、按自增主键做游标翻页，内存占用与
> 导出行数无关）。
> 质量保障上，写了 **234 个后端测试**（状态机、审批链、生命周期、并发、附件、通知、看板等）
> 和 **6 个 Playwright 浏览器端到端脚本共 176 项断言**，覆盖登录、建单、审批、
> 附件上传下载、通知跳转、导出等主流程。

三、系统全景

3.1 架构

```
浏览器 (Vue 3 + Element Plus)
        │  axios + JWT (Authorization: Bearer xxx)
        ▼
Spring Boot 3 (8081)
   ├── JwtAuthenticationFilter   解析 token → 写入 SecurityContext
   ├── SecurityConfig            接口级/角色级授权（进入业务前拦截）
   ├── Controller 层             只接参数、包响应体
   ├── Service 层                业务规则、状态机、审批链、事务
   │     ├── TicketServiceImpl          工单生命周期、审批、催办
   │     ├── ApprovalFlowResolver       审批链解析
```

```
   │      ├── TicketAttachmentServiceImpl 附件校验与存储
   │      ├── NotificationServiceImpl    站内通知写扩散
   │      ├── TicketOpLogServiceImpl     操作审计
   │      └── DashboardServiceImpl       看板聚合
   ├── Mapper 层 (MyBatis-Plus + 注解 SQL)
   └── 全局异常处理 → 统一返回 {code, message, data}
         ▼
MySQL 8 (enterprise_work_order, 10 张表)
本地磁盘 data/attachments (附件按 年/月/日 分目录)
```

3.2 数据模型（10 张表）

表	作用	关键设计
sys_user	用户	角色 EMPLOYEE / APPROVER / ADMIN，BCrypt 存密码
department	部门（树）	parent_id + leader_id，用于"找部门主管"
ticket_type	工单类型	approval_flow 存审批链，如 DEPT_LEADER,ADMIN
ticket	工单主表	状态、当前级次、当前审批人、终态时间
ticket_no_seq	单号序列	每天一行，INSERT ... ON DUPLICATE KEY UPDATE 原子自增
approval_record	审批记录（只追加）	带 step，供程序判定权限与级次
ticket_op_log	操作日志（只追加）	评论/附件/催办也记，供人复盘
ticket_comment	工单讨论	姓名快照，保留历史显示
ticket_attachment	附件	逻辑删除；白名单 + 魔数校验
notification	站内通知	写扩散，一发生就为接收人插行

3.3 工单状态机

```
DRAFT ──提交──► PENDING ──最后一级通过──► APPROVED
  │               │  │
  │               │  └──非最后一级通过──► PENDING (级次 +1)
  │               ├──驳回──► REJECTED ──改完重提──► PENDING
  │               └──撤回──► WITHDRAWN ──改完重提──► PENDING
  │
  ├──删除──► 逻辑删除（草稿专属，从未提交过才能删）
  └──作废──► CLOSED（不可逆终态）

REJECTED / WITHDRAWN ──作废──► CLOSED
```

关键点："能不能改 / 能不能删 / 能不能交 / 能不能审 / 能不能催 / 能不能加附件" 全部写成 `TicketStatus` 枚举上的方法（`isEditable()`、`isDeletable()`…）。 好处是：Service 每个入口只写一句 `status.isEditable()`，规则不会散落两处写岔； 用 `switch` 表达式而不是 `if` 链，将来加新状态编译器会直接报错，逼你补全所有规则。

四、把主流程走一遍（面试时按这个讲）

1. **张三登录**：`POST /api/auth/login` 校验 BCrypt 密码，签发 JWT（HS256，24 小时）。

之后每个请求由 `JwtAuthenticationFilter` 解析出用户，塞进 `SecurityContext`。

2. **张三建单**：选工单类型（类型上挂着审批链配置），保存为 `DRAFT`。

此时生成单号——走 `ticket_no_seq` 的原子自增，避免并发重号。

3. **张三提交**：`TicketServiceImpl.submit()` 调 `ApprovalFlowResolver.resolveChain()`：

把 `DEPT_LEADER,ADMIN` 翻译成具体的人，期间做**自审规避**（审批人不能是提交人）、 **同人去重**（主管恰好也是管理员时只审一次）、**兜底**（部门没配主管就往上级部门找， 找到头还没有就落到管理员）。结果是"第 1 级：李四，第 2 级：管理员"。 状态转 `PENDING`，写一条 `approval_record(step=0, SUBMIT)` 和一条 `ticket_op_log`， 给李四推一条待办通知。

4. **李四审批**：只有 `current_approver_id = 自己` 且状态是 `PENDING` 才能审。

通过则级次 +1、换下一级审批人；已是最后一级则转 `APPROVED`、写 `finished_at`， 通知张三。驳回则转 `REJECTED`，张三可以改完重新提交（重新走一遍审批链）。

5. **中途的沟通**：审批中任何参与人都能评论、上传附件（补发票、补说明），

每一条都进 `ticket_op_log`；申请人还能催办——催办走行锁 + 30 分钟限流， 只推通知、不改流程。

6. **办结之后**：内容、附件全部冻结。附件是"审批时看到的材料"的凭证，

事后若还能改，"当时批的是哪份材料"就说不清了。

7. **横向能力**：通知中心（角标 + 已读）、工作台看板（待办/在审/已通过/未读 +

状态分布 + 7 天趋势）、Excel 全量导出（按当前筛选条件，不是只导当前页）。

五、七个技术亮点（每个都按"问题 → 方案 → 为什么不那样"准备）

亮点 1：单号并发安全

- **问题**：最初用 `SELECT MAX(ticket_no)+1`。读最大值发生在事务提交前，

并发请求读到同一份快照、算出同一个号，第二个插入撞唯一键直接失败。 **实测 20 并发只有 3 个成功。**

- **方案**：新建 `ticket_no_seq(seq_date, current_val)`，用

`INSERT ... ON DUPLICATE KEY UPDATE current_val = current_val + 1` 拿号。这条语句会对该行加排他锁并持有到事务提交，并发被串行化， 每人都拿到不同序号。**锁在数据库，不在 JVM，多实例部署同样安全。**

- **为什么不用**：不用 `UUID`（不可读、无法按天排序）、不用 Redis 自增（本项目还没到需要独立发号器的量级，多引一个组件就多一个故障点）、 不用数据库自增主键（主键不能按天重置，也不该暴露给用户）。
- **会被追问**：这个方案的性能上限在哪？（答：同一天所有建单请求串行在一行上，每天几万单没问题；真到瓶颈再换号段模式，一次领 1000 个号在内存发。）

亮点 2：审批链解析的四个正确性处理

- **问题**：`ticket_type.approval_flow` 存的是角色占位符字符串，但"谁审"要在提交那一刻才算得出来。
- **方案**：`ApprovalFlowResolver` 解析出**完整实际审批人链**，一次性做四件事：

1. **自审规避**：某级审批人就是提交人本人时摘掉该级（主管自己提假的报销单，否则等于自建自批）；
2. **同人去重**：同一个人不在链上出现两次（主管恰好也是管理员，不该让他审两遍）；
3. **向上追溯**：部门没配主管就往上级部门找，最多 10 层（防脏数据成环死循环）；
4. **兜底**：实在找不到就落到管理员；一个人都没有则直接报错，不允许单据进入"没有任何人能审"的假审批中状态。

- **为什么必须"看到整条链"再解析**：逐级解析在第 1 级时根本不知道第 2 级会落到谁头上，去重和自审规避做不对。所以对外只暴露 `resolveChain()`。
- **会被追问**：审批人离职了怎么办？（答：现在没做转办/代理审批，这是明确的下一步；当前设计里管理员可以兜底。）

亮点 3：状态机集中在枚举里

- **问题**：状态规则如果散落在各个 Service 里，迟早在某个入口写漏一条，典型症状是"这个接口允许改被驳回的单、那个接口却不允许"。
- **方案**：所有"能不能"的判断写成 `TicketStatus` 的方法，`switch` 表达式强制覆盖全部枚举常量。
- **关键区分**：`isEditable()` 和 `isAttachable()` 不是同一组状态——`PENDING` 时内容不可改、**附件却可以加**（补材料是审批中最常见的沟通）， 所以它们必须分开写。这就是不能强行复用一个布尔判断的理由。

- **前端配合**：详情接口直接返回 `editable / submittable / withdrawable / cancelable / approvable / urgeable` 这些标记，前端只按布尔值渲染按钮。**状态规则只存在后端一处**，前端不重写——否则两边规则各自演进， 典型症状是后端加了新状态、前端按钮藏起来，功能做了却点不到。

亮点 4：三道权限闸门

层级	位置	管什么	例子
接口级	`SecurityConfig`	哪些接口需要登录	登录和健康检查放行，其余要 token
角色级	`SecurityConfig`	哪些角色能调	审批接口限 ADMIN/APPROVER；类型、用户、部门管理限 ADMIN
数据级	Service	**这一条数据**你能不能动	只能改自己的草稿；只有当前待办人能审；只有创建人能催办

- **为什么角色校验写在配置里而不是 Controller 的 `if` 里**：

配置是"进入业务代码之前"的闸门，角色不符不产生任何副作用； 写在 Controller 里等于已经在业务逻辑和事务之内，防不住误写。

- **为什么数据级不能省**：角色对了不代表这条单归你管——

李四和王五都是审批人，李四不能审王五的待办。

- **为什么 JWT 过滤器要 `addFilterBefore`**： 见亮点 5。

亮点 5：JWT 无状态鉴权 + 过滤器顺序

- **方案**：登录签发 HS256 的 JWT，服务端不存 Session（`SessionCreationPolicy.STATELESS`），多实例部署不用做会话共享。
- **踩过的坑**：JWT 过滤器必须用 `addFilterBefore` 插到 `UsernamePasswordAuthenticationFilter` 之前。若只靠 `@Component` 让 Spring Boot 自动注册，它会落在 Servlet 过滤器链上、执行时机晚于 `AuthorizationFilter`， 授权环节读不到身份，**所有接口 401**。
- **为什么关 CSRF**：前后端分离 + token 放在 Authorization 头，

不依赖 Cookie 自动携带，不存在 CSRF 利用场景。

- **会被追问**：token 过期怎么办？（答：前端 axios 拦截器收到 401 清登录态、跳登录页并记录来路；后端 24 小时过期。没做 refresh token， 因为内部系统的体验收益不值得多一套令牌轮换逻辑。）
- **会被追问**：token 泄露怎么办？（答：HTTPS + 短有效期 + 预留黑名单；

当前是学习项目，没做。）

亮点 6：审计日志与审批记录分开建模

- **问题**：已经有 `approval_record` 了，为什么还要 `ticket_op_log`？
- **答**：两张表分工不同。
  - `approval_record` 带 `step`（第几级），是**给程序算状态**用的——

可见性判定和级次计算都依赖它；

  - `ticket_op_log` 是**给人看历史**用的，覆盖评论、附件增删、催办这些审批链之外的动作。
- **为什么不能合并**：把评论塞进 `approval_record` 会有两个后果——

一是评论的 `step` 只能填 0，破坏"step=0 表示提交"的隐含约定， 任何按 step 过滤的统计都会算错；二是那张表参与"谁有权看这张单"的判定， 往里插一条评论等于把评论人提升成"历史审批人"，凭空获得查看权。

- **为什么审计表没有 `deleted` / `updated_at`**：只追加不修改。

审计记录一旦能改能删，就不再是审计记录了。

- **为什么 `operator_id` 允许 NULL**：为将来的系统动作（如定时自动关闭超期单）留位置，

用 NULL 表示"系统"，比编造 0 或 -1 更诚实。

亮点 7：大数据量流式导出

- **问题**：Excel 导出如果直接把整张表放内存，10 万行会 OOM。
- **方案**：

1. 用 POI 的 **SXSSF** 而不是 XSSF：内存里只保留最近 200 行，

更早的行刷到磁盘临时文件，内存占用与总行数无关； 最后必须调 `dispose()` 清理临时文件。

2. **`flushTo(out)` 与 `close()` 分开**：参数校验、权限校验、数据库异常都发生在

`flushTo` 之前，此时响应流还没被碰过，全局异常处理器能把错误正常转成 JSON。 如果一上来就把 `response.getOutputStream()` 传给 workbook， 一旦出错只能返回半截 zip，前端只看到"请求失败"，真正原因丢在服务端日志里。

3. **游标翻页用自增 `id` 而不是 `(created_at, id)`**：`created_at` 在批量数据里

大量重复且没有索引，用它做游标会退化；`id` 唯一且单调，天然适合。

4. **导出上限配置化**（默认 10 万行），超限只导最新 N 行并在首行写提示。

设上限不是保护数据库，是不让一次误操作生成几百 MB 文件卡死浏览器。

- **为什么不用 EasyExcel**：它本质是 POI 的封装，会引入额外概念和版本约束

（EasyExcel 3.x 锁 POI 4.x，而 POI 4.x 在 JDK 17 上会被反射模块检查拦下）。 只导一张固定表头，直接用 POI 更可控。

- **会被追问**：导出 100 万行怎么办？（答：现在靠上限截断 + 提示；

再往上应该改成异步导出——提交任务、后台生成、完成后发通知给下载链接。）

六、工程化与测试

类型	内容	数量
后端测试	状态机、审批链解析、生命周期、并发单号、附件、评论、通知、看板、权限	**234 个，全绿**
浏览器端到端	主流程冒烟、通知/评论/看板、附件（详情页）、附件（新建/修改页）、操作动态与催办、Excel 导出	**6 个脚本，176 项断言**
数据库	全量 `schema.sql` + 6 个增量迁移脚本 + 演示数据恢复脚本 + e2e 清理脚本	—
接口文档	springdoc-openapi，Swagger UI 可直接调	45 个接口

测试里值得一说的两个做法：

1. **断言用独立 SQL 算期望值，不复用被测代码**。比如"最久等待天数"用例，

期望值由一条独立 SQL 算出。如果直接调 Service 的方法取期望， 等于拿答案验答案——Service 里把"向上取整"改成"向下取整"，用例会跟着一起变， 永远不报错。

2. **端到端脚本跑真实的浏览器**，不是只调接口。附件上传要验证

"下载下来的文件名和内容与上传的完全一致"，通知要验证"点通知能跳到对应工单详情"， 401 要验证"最终停在登录页而不是卡在工单页"。

七、面试复盘：高频问题与参考答法

A. 项目整体类

Q1：简单介绍一下这个项目。

> 一句话：企业内部工单流转与审批平台，员工提单、按类型配置多级审批链、
>
> 审批人处理待办，配通知、评论、附件、审计、看板和 Excel 导出。
>
> 后端 Spring Boot 3 + MyBatis-Plus + MySQL + JWT，前端 Vue 3 + Element Plus，
>
> 10 张表 45 个接口，我独立完成设计与实现，写了 234 个后端测试和 6 个浏览器端到端脚本。

Q2：这个项目最难的地方是什么？

> 三个：一是审批链要"可配置"又要在提交那一刻算准具体审批人，

> 还要处理自审、同人去重、主管缺失；二是并发下单号重复，
> 实测 20 并发只有 3 个成功；三是权限要分三层，尤其数据级——
> 都是审批人，但李四不能审王五的待办。

Q3：为什么做这个项目？

> 工单审批是企业里真实存在、规则明确、又天然带流程和权限的场景，
> 比增删改查更能练到状态机、并发、审计这些工程问题。

Q4：如果重做一次，你会改什么？

> 三件事：一是引入 Flyway 管理数据库迁移（现在靠编号 SQL 手工执行，
> 团队协作容易漏）；二是加 Docker Compose 一键起环境；
> 三是把审批人解析从"提交时算"改成"配置变更时校验 + 提交时算"结合，
> 让配错审批链在保存类型时就报错，而不是等有人提单才暴露。

B. 技术细节类（很可能被追着问）

Q5：审批流怎么设计的？加一级审批要改哪些地方？

> `ticket_type.approval_flow` 是逗号分隔的角色占位符，比如 `DEPT_LEADER,ADMIN`
> 表示两级。加一级只改配置数据，不用改代码；代码侧 `ApprovalFlowResolver`
> 解析成具体人，`ticket.current_step / total_step` 控制进度。

Q6：自己提交的单，自己又是部门主管，会发生什么？

> 解析时会把"第 1 级审批人 = 提交人"那一级摘掉。不能让主管自建自批，
> 否则审批这个动作就失去意义。摘掉后如果链空了，会兜底到管理员；
> 再没有就报错，不允许进入没人能审的状态。

Q7：驳回之后重新提交，审批流怎么走？

> 从第 1 级重新走一遍。因为驳回意味着内容会改，改完的单必须重新获得每一级的确认。
> 每次提交都会写一条 `step=0` 的 `SUBMIT` 记录，所以历史是可追溯的。

Q8：并发下单号怎么保证不重复？

> 用 `ticket_no_seq` 表 + `INSERT ... ON DUPLICATE KEY UPDATE` 原子自增。
> 这条语句对当天那一行加排他锁并持有到事务提交，并发被串行化。
> 之前 `MAX+1` 的写法 20 并发只有 3 个成功，换掉之后全部成功。

Q9：催办的 30 分钟限流怎么做的？会不会有并发漏洞？

> 催办时先 `SELECT ... FOR UPDATE` 拿工单行锁，再查最近一次催办时间判断
> 是否超过 30 分钟。不拿锁的话两个并发请求会同时读到"上次是 40 分钟前"

> 双双通过，限流形同虚设。行锁把同一张单的操作串行化。
>
> 判断用 `< 30` 而不是 `<= 30`：恰好等满 30 分钟应该放行。

Q10：为什么有 approval_record 还要 ticket_op_log？

> 见亮点 6。一句话：一份给程序算状态（带 step、参与权限判定），
>
> 一份给人看历史（含评论、附件、催办）。合并会破坏 step 语义，
>
> 还会让评论人凭空获得查看权。

Q11：权限是怎么做的？

> 三层：接口级（要不要登录）、角色级（哪些角色能调，写在 SecurityConfig）、
>
> 数据级（这条数据归不归你，写在 Service）。
>
> 角色校验放配置是为了在进入业务代码和事务之前就拦掉。

Q12：JWT 相比 Session 的取舍？

> 好处是无状态、多实例不用共享会话；代价是服务端不能主动失效、
>
> token 泄露在有效期内都能用、每次请求都要解析和验签。
>
> 内部系统、有效期 24 小时，这个取舍可以接受。要做主动登出就得加黑名单
>
> 或短 token + refresh token。

Q13：附件上传做了哪些安全校验？

> 三重：扩展名白名单（黑名单永远列不全）、文件魔数校验
>
> （防止把 exe 改名成 pdf）、大小限制（业务层 20MB，
>
> Spring 层刻意放宽到 25MB，让 20~25MB 的请求能进业务层给出带具体数字的提示）。
>
> 文件存磁盘时用 UUID 重命名，避免路径穿越和重名。
>
> **已知不足**：删除附件只删数据库记录，物理文件还在磁盘上，没做清理。

Q14：为什么附件在办结后要冻结？

> 附件是"审批时看到的材料"的凭证。APPROVED 之后还能改，
>
> 事后复盘就说不清"审批人当时依据的是哪份材料"。
>
> 但审批中必须允许增删——让申请人补一张发票是最常见的沟通。

Q15：Excel 导出 10 万行怎么不 OOM？

> SXSSF 流式写，内存只保留最近 200 行；游标翻页用自增 id；
>
> `flushTo` 和 `close` 分开，保证异常时还能返回标准 JSON 错误。
>
> 详细见亮点 7。

Q16：数据库索引是怎么考虑的？

> 按真实查询建：

> - `ticket`：`idx_creator`、`idx_status`、`idx_approver(current_approver_id, status)`；
> - `approval_record`：`idx_approver(approver_id, action)`——"我审过的"这个页签要按操作人过滤，没这个索引是全表扫描；
> - `notification`：`idx_user_read(user_id, is_read, id)`——顶栏未读角标每次进页面都要查，过滤列 user_id 打头，is_read 第二列让未读计数也走这条索引。
> - `ticket_type.code` 用"只约束未删除行"的唯一索引，否则逻辑删除的行会一直占着编码。

Q17：逻辑删除怎么处理的？

> MyBatis-Plus 全局配 `deleted` 字段，查询自动带 `deleted=0`。
> 工单只有从未提交过的草稿能逻辑删除；进过审批流的必须保留，
> 否则 `approval_record` 会变成无主数据。
> 唯一索引要注意配合逻辑删除——所以类型编码用了函数索引只约束未删除行。

Q18：测试怎么保证质量？

> 234 个后端测试覆盖状态机、审批链、生命周期、并发、附件、通知、看板；
> 6 个 Playwright 脚本用真实浏览器跑主流程。
> 有个原则：期望值用独立 SQL 算，不复用被测代码，否则就是拿答案验答案。

Q19：前端怎么做权限？

> 路由守卫做粗粒度（未登录跳登录、角色不够不让进管理页），
> 按钮显隐直接用后端返回的 `editable/approvable/urgeable` 标记。
> 前端不重写状态规则——判断逻辑放两处，早晚会不一致；
> 真正的拦截始终在后端，前端只是体验优化。

Q20：这个项目有什么不足？（这题一定要提前准备，主动承认比被问穿好。）

> 一、没上 Docker 和 CI，环境搭建靠文档；
> 二、附件删除只删记录不删物理文件，需要定时任务清理孤儿文件；
> 三、没有审批人转办/代理，审批人离职需要管理员干预；
> 四、没做批量审批，待办只能一条条处理；
> 五、没有 Redis/消息队列——当前量级不需要，但通知是同步写库的，
> 量大了应该异步化；
> 六、前端打包产物约 865KB 没做分包。

八、简历怎么写

推荐写法（3~4 条，每条都带数字或取舍）：

> **企业工单与审批系统**（个人项目 | Spring Boot 3 + MyBatis-Plus + MySQL + Vue 3）
> - 独立设计并实现工单全生命周期（草稿→多级审批→驳回/撤回/作废→办结），
> 后端 10 张表 45 个接口，前端 9 个页面，代码约 1 万行；
> - 设计基于角色占位符的可配置审批链解析器，支持自审规避、同人去重、
> 部门主管向上追溯与管理员兜底，解决"主管自建自批""审批人中立性"等正确性问题；
> - 定位并修复并发下单号重复缺陷（MAX+1 读快照冲突，20 并发仅 3 成功），
> 改用数据库原子自增序列表，并补充并发回归测试；
> - 基于 Spring Security 实现接口级/角色级/数据级三层权限；
> 用 POI SXSSF 实现内存占用与行数无关的全量 Excel 导出；
> - 编写 234 个后端测试与 6 个 Playwright 端到端脚本（176 项断言），覆盖主流程与边界。

不要写："熟悉 Spring Boot、MyBatis、MySQL、Vue"这种技能罗列—— 面试官看不出你做了什么，只会在简历上找项目里的数字和取舍。

九、自我介绍里的两个版本（背熟）

30 秒版：

> 我做的项目是企业工单与审批系统。员工提单后按工单类型走配置好的多级审批，
> 审批人可以处理待办，申请人能催办、评论、传附件，办结后数据冻结。
> 技术上我比较关注两类问题：一是审批流和权限的正确性，
> 二是并发和数据一致性。后端写了 234 个测试，还用 Playwright 做了浏览器端的流程验证。

2 分钟版：

> 这个项目解决的是企业里"一件事从提出到批准"的流转问题。
> 核心是工单状态机和审批链：工单有草稿、审批中、已通过、已驳回、已撤回、已作废六个状态，
> 每个状态能做什么操作都集中在状态枚举里，避免规则散落。
> 审批链存在工单类型上，用角色占位符配置，提交时解析成具体审批人，
> 过程中要处理审批人就是提交人、同一个人重复出现、部门没配主管这几种情况。
> 工程上有几个点我印象比较深：第一，单号最初用 MAX+1，并发测试 20 个只有 3 个成功，
> 后来改成数据库原子自增序列表解决；第二，权限分了三层，
> 因为都是审批人，但只能审分给自己的单；第三，
> 审计日志和审批记录分成两张表，因为一张要带级次给程序算状态，
> 另一张要记录评论、附件、催办这些给人看的历史，合并会破坏语义。
> 测试上我写了 234 个后端测试和 6 个端到端脚本，

> 端到端脚本是真实浏览器跑的，能验证下载文件的文件名和内容都正确。
>
> 项目现在的不足是没有容器化部署和 CI，附件删除后物理文件还没清理，这些是我下一步要补的。

十、面试前 3 天冲刺清单

- □ 能不看文档画出 10 张表的关系和工单状态机
- □ 能讲清审批链解析的四个处理（自审、去重、向上追溯、兜底）
- □ 能复述单号并发问题：原来错在哪、怎么发现的、怎么改的、结果如何
- □ 能说清三层权限分别拦什么、为什么数据级不能省
- □ 能解释 approval_record 与 ticket_op_log 的分工，以及合并的后果
- □ 能讲 SXSSF 导出：为什么流式、为什么 flushTo 和 close 分开、游标为什么用 id
- □ 准备 3 个"我踩过的坑"故事（单号并发、过滤器顺序、催办限流要加锁）
- □ 准备 6 条不足，每条都带上"如果做我会怎么做"
- □ 把项目跑起来，准备现场演示：登录 → 建单 → 提交 → 切换审批人 → 审批 → 看通知和操作动态
- □ 演示账号：admin / zhangsan / lisi / wangwu，密码都是 123456

演示网址：前端 http://localhost:5180 ，后端 Swagger http://localhost:8081/swagger-ui.html

十一、下一步可以继续完善的功能（按性价比排序）

优先级	功能	面试加分点
高	Flyway 数据库迁移	体现版本管理与团队协作意识
高	Docker Compose 一键起 MySQL + 后端	体现部署能力，直接解决当前短板
中	附件删除时清理物理文件（或定时任务扫孤儿文件）	把"已知缺陷"变成"已解决"
中	审批人转办 / 代理审批	真实业务里必有的能力
中	我的待办批量审批	体现批量操作与事务边界思考
低	数据统计报表（按人/部门/类型）	体现聚合查询与 SQL 能力
低	前端打包分包 + 路由懒加载优化	体现性能意识

附：项目关键信息速查

项	值
后端	`C:\Users\11619\IdeaProjects\enterprise-work-order`，端口 8081
前端	`C:\Users\11619\IdeaProjects\enterprise-work-`

	`order-web`，端口 5180
数据库	MySQL 8，库名 `enterprise_work_order`，10 张表
后端栈	JDK 17、Spring Boot 3.3.5、MyBatis-Plus 3.5.9、jjwt 0.12.6、POI 5.2.5
前端栈	Vue 3.5、Vite 8、Element Plus 2.14、Pinia 4、Vue Router 5、axios
接口数	45 个 REST 接口，Swagger UI 可调
测试	后端 234 个测试全绿；Playwright 6 个脚本 176 项断言
演示账号	admin / zhangsan / lisi / wangwu，密码 123456